



COURSE CODE/TITLE:	CPE	SECTION:	CPE21S1
DATE PERFORMED:	04/08/26	DATE SUBMITTED:	11/08/2026
NAME:	Duenas Ralph Florenz F.	INSTRUCTOR:	Engr. Jimlord Quejado

ACTIVITY NO. ____ [ACTIVITY NAME]

1. PROCEDURE OUTPUT

- 1. ILOA

```
1  #include <iostream>
2  #include <queue>
3
4
5  void display(std::queue<char> r);
6
7  int main(){
8
9
10     std::queue<char> q;
11     q.push('c');
12     q.push('p');
13     q.push('e');
14     q.push('0');
15     q.push('1');
16     q.push('0');
17
18
19
20
21     //DISPLAY
22     display(q);
23
24     //empty()
25     std::cout << "is the function empty? " << q.empty();
26     std::cout << "\n";
27     //size()
28     std::cout << "size of the queue: " << q.size();
29     std::cout << "\n";
30     //front
31     std::cout << "function frot: " << q.front();
32     std::cout << "\n";
33     //back
34     std::cout << "is the function empty? " << q.empty();
35     std::cout << "\n";
```

```

36 //pop
37 q.pop();
38 //display
39 display(q);
40 //push another element
41 q.push('r');
42 q.push('a');
43 q.push('l');
44 q.push('p');
45 q.push('h');
46 q.push('x');
47 //display
48 display(q);
49
50 }

```

```

53 void display(std::queue<char> r){
54 //create a copy of the queue
55 std::queue<char> c = r;
56
57
58 while(!c.empty()){
59     std::cout<< " " << c.front();
60     c.pop();
61
62
63 }
64 std::cout << "\n";
65
66
67 }

```

```

c p e 0 1 0
is the function empty? 0
size of the queue: 6
function frot: c
is the function empty? 0
p e 0 1 0
p e 0 1 0 r a l p h x

```

- 2. ILOB
MAIN

```
5.1 > C:\ilobmain.cpp > main()
1  #include <iostream>
2  #include "queue_ll.h"
3
4  int main()
5
6
7      qNode<char> *front = nullptr;
8      qNode<char> *back = nullptr;
9
10
11     enqueue(&front, &back, 'J');
12     std::cout << front->data << " " << back->data << std::endl;
13
14
15     enqueue(&front, &back, 'I');
16     std::cout << front->data << " " << back->data << std::endl;
17
18
19     enqueue(&front, &back, 'M');
20     std::cout << front->data << " " << back->data << std::endl;
21
22
23     dequeue(&front, &back);
24     std::cout << front->data << " " << back->data << std::endl;
25
26     return 0;
27
```

QUEUE

```

1  #ifndef QUEUE_H
2  #define QUEUE_H
3
4  template <typename T>
5  class qNode {
6  public:
7      T data;
8      qNode* next;
9  };
10
11  //creating a new node
12  template <typename T>
13  qNode<T>* new_node(T newData) {
14      qNode<T>* newNode = new qNode<T>;
15      newNode->data = newData;
16      newNode->next = nullptr;
17      return newNode;
18  }
19
20  template <typename T>
21  void enqueue(qNode<T> **frontPtr, qNode<T> **backPtr, T newData) {
22      //create a new node
23      qNode<T> *newNode = new_node<T>(newData);
24
25      //inserting to an empty queue
26      if ((*frontPtr) == nullptr && (*backPtr) == nullptr) {
27          (*frontPtr) = newNode;
28          (*backPtr) = newNode;
29      }
30
31      //inserting an item into a non empty queue
32      (*backPtr)->next = newNode; //point the backPtr next to the newNode
33      (*backPtr) = newNode;
34  }
35
36  template <typename T>
37  void dequeue(qNode<T> **frontPtr, qNode<T> **backPtr) {

```

```

38 //create a temporary node to store the node to be deleted
39 qNode<T> *deleteNode = nullptr;
40 deleteNode = (*frontPtr);
41
42 //check if the queue is only 1 node
43 if ((*frontPtr)->next == nullptr && (*backPtr)->next == nullptr) {
44     (*frontPtr) = nullptr;
45     (*backPtr) = nullptr;
46     delete deleteNode;
47     return;
48 }
49
50 //deleting of the node
51 (*frontPtr) = deleteNode->next;
52 delete deleteNode;
53 }
54
55 //display all elements in the list
56 template <typename T>
57 void display(qNode<T>* front) {
58     //create a temporary node
59     qNode<T> *displayNode = front;
60
61     while (displayNode != nullptr) {
62         std::cout << displayNode->data;
63         if (displayNode->next != nullptr)
64             std::cout << " -> ";
65         displayNode = displayNode->next;
66     }
67
68     std::cout << std::endl;
69 }
70
71 }

```

```

72
73 //return the front variable
74
75 #endif

```

```

J J
J I
J M
I M

```

- Analysis of the procedure

In the main procedure or main.cpp, the program demonstrated both standard library and custom linked-list queue implementations. It first initialized a standard queue `q` using `std::queue<char>` and inserted characters ('c', 'p', 'e', 'o', 'l', 'o') using `q.push()`. It printed the queue using `display()`, checked if it was empty with `q.empty()`, checked its size with `q.size()`, and displayed the front character using `q.front()`. The program then removed the front element using `q.pop()`, added new characters ('r', 'a', 'l', 'p', 'h', 'x') with `q.push()`, and printed the updated queue using `display()`. Next, it tested a custom linked-list queue by initializing front and back pointers to `nullptr`. It used `enqueue()` to insert nodes containing 'J', 'I', and 'M', printing the updated front and back pointers after each step. Lastly, deletion was demonstrated using `dequeue()` to remove the front node 'J', updating the front pointer to point to the next node ('I') and displaying the final front and back values.

- 2. ILOBP2 MAIN

```

1  #include <iostream>
2  #include "queueArr.h"
3
4  int main(){
5
6      //create an objecy
7
8      //enqueue
9      queueArr<int> q(6);
10     q.Enqueue(1);
11     std::cout << "q_front: " << q.Front() << ", q_back: " << q.Back() << ", q_size: " << q.size() << std::endl;
12
13     q.Enqueue(2);
14     std::cout << "q_front: " << q.Front() << ", q_back: " << q.Back() << ", q_size: " << q.size() << std::endl;
15
16     q.Enqueue(3);
17     std::cout << "q_front: " << q.Front() << ", q_back: " << q.Back() << ", q_size: " << q.size() << std::endl;
18
19     q.Enqueue(4);
20     std::cout << "q_front: " << q.Front() << ", q_back: " << q.Back() << ", q_size: " << q.size() << std::endl;
21
22     q.Enqueue(5);
23     std::cout << "q_front: " << q.Front() << ", q_back: " << q.Back() << ", q_size: " << q.size() << std::endl;
24
25     q.Enqueue(6);
26     std::cout << "q_front: " << q.Front() << ", q_back: " << q.Back() << ", q_size: " << q.size() << std::endl;
27
28     // dequeue
29     q.Dequeue();
30     std::cout << q.Front() << ", q_back: " << q.Back() << ", q_size: " << q.size() << std::endl;
31
32     q.Dequeue();
33     std::cout << q.Front() << ", q_back: " << q.Back() << ", q_size: " << q.size() << std::endl;
34
35     q.Dequeue();
36     std::cout << q.Front() << ", q_back: " << q.Back() << ", q_size: " << q.size() << std::endl;
37
38     q.Dequeue();
39     std::cout << q.Front() << ", q_back: " << q.Back() << ", q_size: " << q.size() << std::endl;
40
41     q.Dequeue();
42     std::cout << q.Front() << ", q_back: " << q.Back() << ", q_size: " << q.size() << std::endl;
43
44     q.Dequeue();
45     std::cout << q.Front() << ", q_back: " << q.Back() << ", q_size: " << q.size() << std::endl;
46
47     q.Dequeue();
48     std::cout << q.Front() << ", q_back: " << q.Back() << ", q_size: " << q.size() << std::endl;
49     return 0;
50
51     // copy constructor
52     queueArr<int> q2(q);
53     std::cout << "q2_front: " << q2.Front() << ", q2_back: " << q2.Back() << ", q2_size: " << q2.size() << std::endl;
54
55     // assignment operator
56     queueArr<int> q3(6);
57     q3 = q;
58     std::cout << "q3_front: " << q3.Front() << ", q3_back: " << q3.Back() << ", q3_size: " << q3.size() << std::endl;
59
60

```

ARR QUEUE

```
1  #ifndef QUEUEARR_H
2  #define QUEUEARR_H
3
4  #include <iostream>
5  template <typename T>
6  class queueArr{
7  private:
8      // create a pointer array:
9      T* q_array;
10     size_t q_capacity;
11     size_t q_size;
12     int q_front;
13     int q_back;
14
15
16 public:
17     //prototypes to use for queueArr:
18     queueArr(size_t capacity);
19     bool empty();
20     bool full();
21
22     size_t size();
23     void Clear();
24
25     T Front();
26     T Back();
27
28     void Enqueue(T newData);
29     void Dequeue();
30
31     ~queueArr();
32
33
34
35     //tasks
36     //copy constructor
37     queueArr(const queueArr& other);
38
39     //assignment operator
40     queueArr& operator=(const queueArr& other);
41 };
```

```
42
43 // constructor
44 template <typename T>
45 queueArr<T>::queueArr(size_t capacity){
46
47     //capacity of the array
48     q_capacity = capacity;
49
50     //current size of the array
51     q_size = 0;
52
53     //initial front and back of the index
54     q_front = 0;
55     q_back = -1;
56
57     // newly allocated array
58     q_array = new T[q_capacity];
59     std::cout << "Queue created with capacity: " << q_capacity << std::endl;
60 }
61
62 //destructor
63 template <typename T>
64 queueArr<T>::~queueArr(){
65
66     delete [] q_array;
67     std::cout << "Queue destroyed." << std::endl;
68 }
69
70
71 //empty
72 template <typename T>
73 bool queueArr<T>::empty(){
74     return q_size == 0;
75 }
76
77 //full
78 template <typename T>
79 bool queueArr<T>::full(){
80     return q_size == q_capacity;
81 }
82
83 //clear
84 template <typename T>
85 void queueArr<T>::clear(){
86     q_size = 0;
87     q_front = 0;
88     q_back = -1;
89 }
```



```

90
91 //return the element in the front of the queue
92 template <typename T>
93 T queueArr<T>::Front(){
94     //check and display if the queue is empty
95     if (empty()){
96         std::cout << "Queue is empty. \n" << std::endl;
97         return T{}; // return default value of T
98     }
99     //return the front element
100     return q_array[q_front];
101 }
102
103 // back
104 template <typename T>
105 T queueArr<T>::Back(){
106     //check and display if the queue is empty
107     if (empty()){
108         std::cout << "Queue is empty. \n" << std::endl;
109         return T{}; // return default value of T
110     }
111     // if not then return the back element
112     return q_array[q_back];
113 }
114
115 template <typename T>
116 size_t queueArr<T>::size(){
117     return q_size;
118 }
119
120 template <typename T>
121 void queueArr<T>::Enqueue(T newData){
122     // check if the queue is full
123     if (full()){
124         std::cout << "Queue is full." << std::endl;
125         return;
126     }
127
128     // compute for the q_back to move it circularly
129     q_back = (q_back + 1) % q_capacity;
130
131     //assign the new value to the back of the queue array
132     q_array[q_back] = newData;
133     //increment the q_size
134     q_size++;
135 }

```

```

136
137 //dequeue
138 template <typename T>
139 void queueArr<T>::Dequeue(){
140     // check if the queue is empty
141     if (empty()){
142         std::cout << "Queue is empty." << std::endl;
143         return;
144     }
145
146     // compute for q_front to move it circularly
147     q_front = (q_front + 1) % q_capacity;
148
149     //decrement the size q_size
150     q_size--;
151 }

```

```

152
153 // copy constructor
154 template <typename T>
155 queueArr<T>::queueArr(const queueArr& other) {
156     q_capacity = other.q_capacity;
157     q_size = other.q_size;
158     q_front = other.q_front;
159     q_back = other.q_back;
160
161     // Allocate new memory for the array
162     q_array = new T[q_capacity];
163
164     // Copy the elements from the other queue
165     for (size_t i = 0; i < q_size; ++i) {
166         size_t index = (q_front + i) % q_capacity;
167         q_array[index] = other.q_array[index];
168     }
169 }
170
171 // copy assignment operator
172 template <typename T>
173 queueArr<T>& queueArr<T>::operator=(const queueArr& other) {
174     if (this != &other) {
175         // Clean up existing memory
176         delete [] q_array;
177
178         // Copy properties from the other queue
179         q_capacity = other.q_capacity;
180         q_size = other.q_size;
181         q_front = other.q_front;
182         q_back = other.q_back;
183
184         // Allocate new memory for the array
185         q_array = new T[q_capacity];
186
187         // Copy the elements from the other queue
188         for (size_t i = 0; i < q_size; ++i) {
189             size_t index = (q_front + i) % q_capacity;
190             q_array[index] = other.q_array[index];
191         }
192     }
193     return *this;
194 }
195 #endif

```

```

Queue created with capacity: 6
q_front: 1, q_back: 1, q_size: 1
q_front: 1, q_back: 2, q_size: 2
q_front: 1, q_back: 3, q_size: 3
q_front: 1, q_back: 4, q_size: 4
q_front: 1, q_back: 5, q_size: 5
q_front: 1, q_back: 6, q_size: 6
2, q_back: 6, q_size: 5
3, q_back: 6, q_size: 4
4, q_back: 6, q_size: 3
5, q_back: 6, q_size: 2
6, q_back: 6, q_size: 1
Queue is empty.

```

```
0, q_back: Queue is empty.

0, q_size: 0
Queue is empty.
Queue is empty.

0, q_back: Queue is empty.

0, q_size: 0
Queue destroyed.
```

- ANALYSIS OF THE PROCEDURE

In the main procedure or main.cpp, the program demonstrated the functionality of a custom array-based circular queue using the queueArr class. It first initialized a queue object q with a capacity of 6 using queueArr<int> q(6). The program then inserted integers from 1 to 6 using q.Enqueue(), printing the queue's front element with q.Front(), back element with q.Back(), and total size with q.size() after every addition. Next, it tested element removal by calling q.Dequeue() seven consecutive times, displaying the updated front, back, and size values after each removal step to monitor how the queue emptied. Finally, the program verified object copying by using the copy constructor to initialize a second queue q2 from q (queueArr<int> q2(q);) and the assignment operator to assign q to a third queue q3 (q3 = q;), outputting each queue's front, back, and size values to confirm that the dynamic data was duplicated correctly.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

```
1  #include <iostream>
2  #include <string>
3
4
5  // create a class called job
6  class Job{
7      private:
8          // Id number for the job
9          int identification;
10         // name of the user who submitted the job
11         std::string name;
12         //number of pages for the printer
13         int pages;
14
15     public:
16         // constructor
17         Job() {
18             identification = 0;
19             name = "";
20             pages = 0;
21         }
22
23
24         // constructor for the job class
25         Job(int i, std::string n, int p) {
26             identification = i;
27             name = n;
28             pages = p;
29         }
30
31
32         // function to print out the job's details
33         void display() {
34             std::cout << "Input Job ID: " << identification << ", Input Name: " << name << ", Input Pages: " << pages << std::endl;
35         }
36     };
37 }
```

```
40 class Printer{
41     private:
42         const int capacity = 10;
43         Job jobs[10];
44         int front;
45         int rear;
46         int count;
47
48
49     public:
50         // create a queue to hold the jobs
51         Printer() {
52             front = 0;
53             rear = -1;
54             count = 0;
55         }
56         // checks if the queue is full
57         bool isFull() {
58             if (count == capacity) {
59                 return true;
60             }
61             return false;
62         }
63         // Check if queue is empty
64         bool isEmpty() {
65             if (count == 0) {
66                 return true;
67             }
68             return false;
69         }
}
```

```

70 // function to add a job to the queue
71 void addJob(Job newJob){
72     if (isFull()) {
73         std::cout << "Printer queue is full." << std::endl;
74         return;
75     }
76     rear = (rear + 1) % capacity; // this moves the rear index circularly
77     jobs[rear] = newJob;
78     count++;
79     std::cout << "Job added to the printer queue." << std::endl;
80 }
81 // function to process the job from the queue
82 void processJob() {
83     std::cout << "\nProcessing job:\n" << std::endl;
84
85     while (!isEmpty()) {
86         std::cout << "Printing" << std::endl;
87         jobs[front].display();
88         front = (front + 1) % capacity; // this moves the front index circularly
89         count--;
90         std::cout << std::endl;
91     }
92 }
93 // shows the waiting line for the jobs awaiting to be processed
94 void showQueue() {
95     if (isEmpty()) {
96         std::cout << "\nPrinter queue is empty." << std::endl;
97         return;
98     }
99
100
101     std::cout << "\nJobs in the printer queue:" << std::endl;
102     int index = front;
103     for (int i = 0; i < count; i++) {
104         jobs[index].display();
105         index = (index + 1) % capacity;
106     }
107     std::cout << std::endl;
108 }
109 };

```

```
110 int main()
111     //creating of objects for the printer and jobs
112     Printer printer;
113
114
115     int numberOfJobs;
116     std::cout << "Input the number of jobs to add to the printer queue: ";
117     std::cin >> numberOfJobs;
118
119     std::cout << std::endl;
120
121
122     for (int i = 0; i < numberOfJobs; i++) {
123         int numPages;
124         std::string name;
125
126
127         std::cout << "Input User Name: " << (i + 1) << ": ";
128         std::cin >> name;
129         std::cout << "Input Number of Pages: " << (i + 1) << ": ";
130         std::cin >> numPages;
131
132
133         Job newJob(i + 1, name, numPages);
134         printer.addJob(newJob);
135         std::cout << std::endl;
136     }
137
138
139     // Show the jobs in the queue
140     printer.showQueue();
141
142     // Process the jobs in the queue
143     printer.processJob();
144
145     // Show the jobs in the queue after processing
146     printer.showQueue();
147
148
149     return 0;
150 }
```

```

Input User Name: 1: Ralph
Input Number of Pages: 1: 2
Job added to the printer queue.

Input User Name: 2: Drei
Input Number of Pages: 2: 2
Job added to the printer queue.

Jobs in the printer queue:
Input Job ID: 1, Input Name: Ralph, Input Pages: 2
Input Job ID: 2, Input Name: Drei, Input Pages: 2

Processing job:

Printing
Input Job ID: 1, Input Name: Ralph, Input Pages: 2

Printing
Input Job ID: 2, Input Name: Drei, Input Pages: 2

Printer queue is empty.

```

- Summary of Lessons Learned

In this HOA, I learned how to implement and use a queue data structure using both the standard library (`std::queue`) and a custom array-based circular queue (`queueArr`). From my first `main.cpp` code, I learned basic queue operations by pushing characters onto a queue, checking `q.front()`, `q.size()`, and `q.empty()`, and using `q.pop()` to remove elements following the First-In, First-Out (FIFO) rule. I then applied these exact same operations `addJob()`, `processJob()`, and `showQueue()` to solve a real problem: simulating a shared office printer where multiple users submit print requests. Incoming print tasks with job IDs, user names, and page numbers are queued up, and the printer processes them one by one in the exact order they arrived. Using a circular array for the custom queue allows it to reuse array slots efficiently using modulo arithmetic without shifting memory.

- Analysis of Supplementary Activity

In the supplementary activity or `suppmain.cpp`, the program shows a shared printer simulation using a custom array-based queue to handle print jobs in a first-come, first-served order. It first initialized a `Printer` object named `printer` with a capacity of 10 and prompted the user to input the number of print jobs to add. The program then collected job details such as user name and page count—and inserted each new `Job` object into the queue using `printer.addJob(newJob)`, automatically generating job IDs and printing a confirmation after every addition. Next, it displayed the current waiting line using `printer.showQueue()` before processing the tasks. It then called `printer.processJob()`, which looped through the queue to print and dequeue each job one by one while updating the front index circularly and decrementing the job count. Finally, the program verified that all tasks were completed by calling `printer.showQueue()` a second time, outputting that the printer queue was empty to confirm proper execution.

- Concluding Statement / Feedback

I think I did very well in this HOA. I was able to successfully implement the algorithm using a custom array-based queue and correctly simulate a shared printer handling print jobs in FIFO order. However, I still need more practice in handling template functions cleanly and managing dynamic pointers without causing memory errors happening mid-way through execution.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: Copilot

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: Explanation of concept and the flow of the code itself.

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: I used Copilot to elaborate the thoughts I have in my mind without replacing the original thought and also for getting the formulas that are needed. I affirm that my use of AI complies with TIP's AI usage policy.

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.